Day 2 — Backend architecture, concurrency, databases, APIs, and DSA

Outcome

Be able to turn an AI use case into clean service boundaries, stable APIs, appropriate persistence, bounded concurrency, reliable background work, and explainable algorithms.

1. Architecture patterns

Pattern map

Pattern	AI/backend use	Main trade-off
Factory	Select model/provider/tool adapter	Central creation helps consistency; giant factories become conditional dumps.
Strategy	Retrieval, routing, chunking, scoring	Runtime flexibility; too many strategies obscure simple behavior.
Adapter	Normalize provider SDKs/ vector stores	Reduces coupling; lowest-common-denominator interfaces may hide useful features.
Decorator	Retry, logging, metrics, cache	Composable cross-cutting behavior; stacked wrappers can hide operational order.
Facade	Simple API over a RAG subsystem	Convenient boundary; can become a god object.
Repository	Persistence behind domain operations	Testability and separation; unnecessary for trivial data access.

Use layered architecture:

```
API
→ application/service
→ domain interfaces
→ adapters/infrastructure
```

The API layer handles transport, validation, identity context, and response mapping. It should not contain retrieval algorithms, provider SDK code, or database transaction choreography.

Decorator versus middleware

A decorator wraps a function or component and fits focused behavior such as retry, cache, logging, or metrics. Middleware wraps the broader request/agent execution path and fits cross-cutting policies such as authentication, rate limiting, tracing, or PII handling.

Both can hide operational order when stacked carelessly. Make execution order observable and keep business logic out of wrappers.

2. HTTP and API design

Method, request, and status semantics

Method	Meaning
`GET`	Retrieve a resource.
`POST`	Create a resource or trigger an operation.
`PUT`	Replace a resource; normally idempotent.
`PATCH`	Partially update a resource.
`DELETE`	Remove a resource.
`HEAD`	Return metadata without a response body.
`OPTIONS`	Describe supported capabilities/CORS behavior.

A request can carry different kinds of input:

```
POST /v1/conversations/conv-123/messages?stream=true
Authorization: Bearer <token>
Idempotency-Key: abc-123

{"message": "Summarize the uploaded contract"}
```

- `conv-123` is a path parameter.
- `stream=true` is a query parameter.
- identity, content type, idempotency, and request IDs are headers.
- the JSON object is the request body.

Key statuses:

Status	Meaning
`200` / `201` / `202` / `204`	Completed / created / accepted asynchronously / succeeded without a body.
`400` / `422`	Invalid business request / structurally or semantically invalid content.
`401` / `403`	Missing-invalid authentication / authenticated but forbidden.
`404` / `409` / `429`	Not found / state or idempotency conflict / rate limited.
`500` / `502` / `503` / `504`	Internal failure / bad upstream response / unavailable / upstream timeout.

Resource-oriented contracts

Typical long-running AI operations:

```
POST /v1/projects/{project_id}/runs
GET  /v1/runs/{run_id}
POST /v1/runs/{run_id}/cancellation
POST /v1/documents/{document_id}/ingestion
GET  /v1/jobs/{job_id}
POST /v1/feedback
```

Return `202 Accepted` when work is accepted but not complete. Include a run/job identifier and status location.

Idempotency

An idempotency key prevents a repeated client request from creating duplicate work:

```
(tenant_id, endpoint, idempotency_key) → original result
```

Apply uniqueness at a trusted persistent boundary. A retry may arrive after the first request committed but before its response reached the client.

Pagination

- Offset pagination is simple but becomes unstable and costly on large changing datasets.
- Cursor pagination uses a stable sort key, for example `(created_at, run_id)`.

```
Query: last 20 runs for a project
Index: (project_id, created_at DESC, run_id DESC)
```

Filtering and sorting belong to the public contract:

```
GET /v1/projects/proj-123/runs
    ?status=FAILED
    &created_after=2026-07-01T00:00:00Z
    &limit=20
    &cursor=<opaque-cursor>
```

Validate allowed filters, sort keys/directions, tenant ownership, date ranges, and maximum page size. The cursor normally encodes the last stable sort fields, such as `(created_at, run_id)`, but clients should treat it as opaque.

Standard errors

Return stable machine codes plus safe human messages and a request identifier. Do not expose raw exceptions.

API versioning

Additive fields are normally safer than removing or changing existing semantics. Removing a field, changing its type or meaning, or making an optional field required is a breaking change and needs an explicit version/migration process. Track model, prompt, embedding, index, tool, and API versions where behavior depends on them.

Polling, webhooks, and streaming

For long-running work, completion delivery is a separate design decision:

Mechanism	Best fit	Main operational concern
Polling `GET /runs/{id}`	Simplest MVP and moderate traffic	Repeated reads and stale client view.
Webhook	Server-to-server completion notification	Callback authentication, retries, delivery history, consumer outages, and DLQ handling.
SSE	One-way browser status or token events	Reconnection, partial delivery, and connection lifetime.
WebSocket	Truly bidirectional real-time interaction	Stateful connection and scaling complexity.

The database/job record remains the durable status source. A stream or webhook is a delivery mechanism, not proof that the client received or processed the final state.

3. Relational, NoSQL, cache, object, and vector storage

Choose from access patterns.

Store	Good fit
Relational	Transactions, state transitions, tenant ownership, uniqueness, audit queries, approvals.
Document NoSQL	Flexible records with stable partition/access patterns.
Key-value/Redis	Cache, rate limits, transient status, locks, deduplication, short-lived state.
Object storage	Original documents, large prompts/outputs, artifacts, exports, model files.
Vector store	Similarity search over chunk embeddings with metadata filters.
Monitoring/time-series	Metrics and operational trends.

Do not store everything in one database merely for convenience.

Core relational concepts

- Primary key identifies a row.
- Foreign key expresses ownership/relationship.
- Transaction groups changes atomically.
- Index accelerates a specific access path at write/storage cost.
- Normalization reduces duplicated facts; denormalization can accelerate known reads.
- `WHERE` filters rows before aggregation; `HAVING` filters aggregate groups.
- Joins combine related tables.

Typical one-to-many relationships include organization-to-users, conversation-to-messages, and document-to-chunks. Foreign keys protect referential integrity.

ACID gives transactions four properties:

- Atomicity: all writes succeed or none do.
- Consistency: constraints and invariants remain valid.
- Isolation: concurrent transactions interact under defined rules.
- Durability: committed data survives failure.

Isolation still requires deliberate handling of lost updates and similar races through optimistic version checks, row locks, or stronger isolation where the invariant warrants it.

CRUD means create, read, update, and delete. Repository/ORM methods should expose domain-relevant operations rather than leaking arbitrary persistence behavior.

Avoid indexing every column. Prefer indexes tied to frequent queries and ordering.

Example join and aggregation:

```
SELECT d.department, COUNT(*) AS document_count
FROM documents AS d
WHERE d.status = 'approved'
GROUP BY d.department
HAVING COUNT(*) >= 5
ORDER BY document_count DESC;
```

`WHERE` removes rows before grouping; `HAVING` removes aggregate groups afterward. Parameterize user-supplied values rather than assembling SQL strings.

ORM and N+1 behavior

An ORM maps application entities and relationships to relational data. Sessions and transactions define a unit of database work; on failure, roll back rather than leaving a partially applied use case.

The N+1 pattern occurs when one query loads a collection and then one extra query runs for every row:

```
1 conversation query
+ 100 message queries
= 101 queries
```

Use eager/join/select-in loading, explicit batch queries, or projections for the required response. Do not eagerly load unbounded child collections by default.

AI data model starter

```
Tenant
User
Project
AgentRun
ToolExecution
Artifact
Feedback
AuditLog
Document
Chunk
IngestionJob
OutboxEvent
```

Large payloads live in object storage; relational rows keep URIs, sizes, hashes, types, ownership, and versions.

Cache risks

- TTL (time to live) expires transient entries such as cached results, rate-limit windows, temporary status, and deduplication records; expiration does not turn Redis into the durable source of truth.
- A read-through cache loads from the durable store on a miss and populates the cache for later reads. Define invalidation/versioning and failure behavior.
- Stale results.
- Cross-tenant cache keys.
- Cache stampede.
- Treating cache as the only durable store.
- Locks without expiry or ownership.

Security-sensitive cache keys include tenant, permission scope, model/prompt/index versions, and query normalization.

4. Async and concurrency

Mental model

Concurrency means tasks overlap in time. Parallelism means work executes simultaneously.

`async` helps cooperative I/O; it does not make CPU-heavy inference faster.

```
async def retrieve_and_generate(question):
    evidence = await retriever.search(question)
    return await model.generate(question, evidence)
```

At `await`, the coroutine yields control while it waits. Blocking I/O or CPU work that does not yield blocks the event loop.

Coroutine versus task

- Calling an `async def` returns a coroutine object.
- Awaiting it runs cooperatively.
- A task schedules a coroutine for concurrent progress.

Sequential:

```
vector = await vector_search(query)
keyword = await keyword_search(query)
```

Concurrent:

```
vector, keyword = await asyncio.gather(
    vector_search(query),
    keyword_search(query),
)
```

Use concurrency only when operations are independent.

`gather` and `asyncio.TaskGroup`

Understand whether one failure cancels siblings, propagates immediately, or is returned as a result. `asyncio.TaskGroup` gives child tasks a structured lifetime and clearer sibling-failure behavior.

`gather(..., return_exceptions=True)` places failures in the result list; callers must inspect those entries rather than treating every result as success. Prefer a task group when related child operations should share one structured lifetime and sibling failure should be handled as a group.

Timeouts and cancellation

Every external operation needs a deadline. Cancellation must propagate through work where possible, but an external side effect may already have occurred.

Do not swallow cancellation as an ordinary exception. Clean up, preserve the cancelled state, and propagate it according to the task contract.

Bounded concurrency

Use a semaphore or worker limit to protect providers, databases, and memory:

```
semaphore = asyncio.Semaphore(max_concurrency)

async with semaphore:
    return await provider.generate(prompt)
```

More concurrency is not automatically more throughput; provider quotas or database capacity may be the bottleneck.

Async, threads, and processes

Tool	Use
Async	Many cooperative I/O operations.
Threads	Blocking I/O libraries that cannot be made async.
Processes	CPU-heavy work needing parallel execution.
External queue/workers	Durable, retryable, bursty, or long-running jobs.

`concurrent.futures` provides thread and process pools for integrating blocking or CPU-oriented work. Reuse bounded pools; creating a process pool per request is expensive.

Bridge a blocking library deliberately:

```
result = await asyncio.to_thread(blocking_client.fetch, document_id)
```

This keeps the event loop responsive, but it does not make the blocking client thread-safe or remove the need for a bounded pool, timeout, and cancellation strategy.

Background work

Do not rely on an untracked `create_task()` for work that must survive process failure.

If `create_task()` is used for request-local concurrent work, keep a reference, observe exceptions, define cancellation/cleanup, and do not let the request finish while required work is silently orphaned.

Use a queue or durable workflow mechanism when work:

- outlives the HTTP request;
- requires retry/recovery;
- is bursty;
- needs rate control;
- has business state.

Race conditions

Single-threaded async code can race when tasks interleave around `await` and mutate shared state. Prefer isolated state, database atomicity, locks only where necessary, and optimistic concurrency for persistent state.

`asyncio.Lock` provides exclusive access among asyncio tasks using the same in-process lock:

```
async with counter_lock:
    counter += 1
```

It does not coordinate OS threads or separate processes. Prefer message passing, one-owner tasks, database atomic updates, or idempotency before introducing distributed locks.

Distributed locking is subtle: define ownership, expiry, failure recovery, and what happens if a holder pauses after its lease expires. Do not let a lock substitute for database constraints and idempotency.

Async debugging and distributed tracing

Asyncio debug mode can expose slow callbacks, wrong-thread API use, and event-loop stalls during development. Monitor event-loop lag, active calls, semaphore wait, pool saturation, queue depth, cancellations, timeouts, and partial failures.

```
POST /answer
├── permission_lookup
├── vector_search
├── keyword_search
├── reranking
└── generation
```

Concurrent spans should overlap in a distributed trace. This reveals blocking code and accidentally sequential awaits.

5. Reliable execution patterns

Transactional outbox

Avoid:

```
commit run
publish event
```

The database commit can succeed and publication fail.

Instead, one database transaction creates:

```
AgentRun
OutboxEvent
AuditLog
```

A publisher sends pending outbox rows. The consumer remains idempotent because publication may repeat.

At-least-once processing

Treat duplicate delivery as expected:

- event/run/tool IDs;
- unique constraints;
- state-transition validation;
- idempotent external calls;
- deduplication records.

Sagas and compensating actions

A database rollback can undo writes inside its transaction; it cannot undo an external API call, published message, sent notification, or completed benchmark action.

For a multi-step business operation, a saga coordinates committed steps and explicit compensating actions where reversal is valid:

```
reserve capacity
→ start run
→ publish result

failure after reservation
→ cancel run if possible
→ release reserved capacity
→ record final outcome
```

Compensation is a new business action, not a time-reversed transaction. Some effects cannot be fully undone, so prevention, approval, idempotency, and reconciliation may be more important than compensation.

State machine

```
QUEUED → RUNNING
RUNNING → WAITING_FOR_TOOL
WAITING_FOR_TOOL → RUNNING
RUNNING → SUCCEEDED | FAILED | CANCEL_REQUESTED
CANCEL_REQUESTED → CANCELLED
```

Terminal states are normally immutable. Use version checks or conditional updates for concurrent changes.

Failure controls

- Timeout.
- Retry only transient errors.
- Exponential backoff and jitter.
- Maximum attempts/deadline.
- Circuit breaker.
- Bulkheads per workload/provider/tenant.
- Backpressure and admission control.
- Dead-letter queue.
- Reconciliation for stuck states.

6. DSA recognition map

Complexity

Big-O describes how time or space grows with input size. State both time and auxiliary space. Consecutive loops are often $O(n) + O(n) = O(n)$, not $O(n^2)$.

Arrays, strings, hashing, and prefix sums

Use:

- traversal for one-pass aggregation;
- dictionary for counts or key lookup;
- set for membership/deduplication;
- prefix sum for repeated contiguous-range totals;
- prefix sum plus hash map for counting subarrays with a target sum.

A subarray or substring is contiguous. A subsequence preserves order but may skip elements. Sliding-window and prefix-sum techniques solve contiguous ranges, not arbitrary subsequences.

Anagram detection is the frequency-counting pattern: two strings must contain the same symbols with the same counts. A set alone is insufficient because it discards counts. Clarify normalization rules for case, spaces, and character set.

Example: token usage range:

```
prefix = [0]
for value in daily_tokens:
    prefix.append(prefix[-1] + value)

range_total = prefix[right + 1] - prefix[left]
```

Two pointers

Use when pointer movement exploits ordering or when comparing from both ends.

- Two-sum in a sorted array.
- Palindrome.
- Remove duplicates from sorted data.
- Slow/fast pointers for cycle detection in a linked list.

Do not use the sorted-array invariant on unsorted input without accounting for sorting and original indices.

Sliding window

Use for contiguous regions.

- Fixed size: maximum of `k` consecutive values.
- Variable size: longest/shortest region satisfying a condition.

A nested shrinking loop can remain `O(n)` because each pointer advances at most `n` times.

For the longest substring without repeating characters, expand the right pointer, track current characters or last-seen indexes, and move the left pointer past a duplicate. This is a contiguous-window problem, not a subsequence problem.

Sum-based variable windows generally require monotonic behavior such as positive values; negative values can break the shrink logic.

Stack and queue

Stack:

- balanced delimiters;
- parsing;
- DFS;
- monotonic next-greater/smaller patterns;
- undo/execution history.

A monotonic stack keeps values or indexes in increasing/decreasing order so each item is pushed and popped at most once. It supports next-greater/smaller and threshold-span problems; store indexes when distance or position is required.

Balanced-parentheses validation pushes opening symbols, checks before popping, matches bracket types, and succeeds only when the stack is empty at the end.

Queue:

- arrival-order processing;
- BFS;
- worker scheduling.

Use `deque`, not `list.pop(0)`, for an efficient Python queue.

Trees and graphs

Tree terms include root, parent, child, leaf, depth, height, and subtree. A binary tree allows at most two children per node; it is not automatically a binary search tree (BST). A valid BST adds an ordering invariant, and a skewed BST can degrade search from the balanced `O(log n)` case to `O(n)`.

Tree traversals:

- pre-order: node before children;
- in-order: sorted order for a binary search tree;
- post-order: children before node.

Graph representation:

```
graph = {
    "ingest": ["parse"],
    "parse": ["chunk"],
    "chunk": ["embed"],
}
```

An adjacency list is memory-efficient for sparse graphs. A directed graph models one-way dependencies; an undirected graph models symmetric relationships. A directed acyclic graph supports topological scheduling.

DFS explores depth; BFS explores levels and finds shortest paths in unweighted graphs. Mark visited when enqueuing in BFS to avoid duplicates.

Topological ordering schedules a DAG of dependencies. A cycle prevents a complete topological order.

Kahn’s algorithm repeatedly processes zero-indegree nodes and reduces the indegree of their dependents. If fewer than all nodes are processed, the dependency graph contains a cycle. With an adjacency list, BFS, DFS, and topological processing are normally `O(V + E)` time and `O(V)` auxiliary state, excluding the graph itself.

Dynamic programming

Look for:

- overlapping subproblems;
- optimal substructure;
- a repeatable state and transition.

Memoization is top-down caching; tabulation is bottom-up iteration.

0/1 knapsack chooses each item zero or one time. One-dimensional optimization iterates capacity backward so one item is not reused in the same round.

AI/backend connections:

- prompt-context selection under a token budget;
- resource allocation;
- dependency/workflow graphs;
- request windows and rate monitoring;
- deduplicating event IDs.

7. Production example: AgentRun API

Start flow:

```
authenticate
→ derive tenant
→ validate
→ authorize
→ enforce idempotency
→ transaction: run + outbox + audit
→ return 202 and run_id
→ publisher sends event
→ worker advances state machine
```

Status queries use relational storage or a safe cache. Artifacts use object storage. Transient limits/status may use Redis. Tool callbacks require signed requests, replay protection, provider event IDs, and state validation.

Flask versus FastAPI

- Flask is minimal, flexible, mature, and well suited to small or synchronous services; validation and API documentation require extra setup.
- FastAPI uses type hints, Pydantic validation, automatic OpenAPI documentation, and strong async support.

Framework choice is secondary to contracts, testability, observability, security, and operational discipline.

8. Trade-offs

- Modular monolith versus microservices: begin modular; split for scaling, isolation, ownership, or release cadence.
- SQL versus NoSQL: transactions and relationships favor SQL; fixed massive partitioned access may favor NoSQL.
- Queue/workers versus durable engine: simplicity versus persisted workflow semantics and recovery.
- Polling versus webhook versus streaming: client simplicity versus delivery/connection complexity.
- Async versus worker queue: low-latency I/O overlap versus durable background execution.
- Cache versus freshness/security: faster reads versus invalidation and scope correctness.

Project-grounded examples

Scenario 1: parallel benchmark campaigns and statistics collection

Project scenario. DPDK Automation for Network Packet Processing replaced manual runs with a parameter-driven pipeline that could submit 10–50+ scenarios, run multi-server benchmarking in parallel, switch BIOS profiles, execute workloads, and capture selected CPU/system statistics in parallel. Bash scripts collected measurements such as powerstat and turbostat output, Python modules processed the results, custom parsers normalized benchmark logs, and a structured database fed comparison dashboards.

How the concepts apply. This is a real concurrency and backend-orchestration problem, not simply "use `async`." Remote host setup, benchmark execution, packet generation, statistics collection, reboots, parsing, and persistence have different lifetimes and failure modes. Concurrency is valuable between independent servers or collectors, while ordering is essential inside a scenario—for example, the required environment and BIOS state must exist before a valid benchmark run.

Decision and trade-offs. Parallel multi-server execution made large campaigns practical, but it increased coordination, partial-failure, resource-contention, and result-correlation risk. Parameterized templates improved repeatability, while retaining more than ten workload variables preserved experimental flexibility. The trade-off was a larger validation surface: defaults reduced user error, but the framework still had to allow expert overrides.

Senior/Staff interview framing.

- **Senior:** draw one scenario as a stateful sequence—configure, reboot where required, verify, run benchmark and collectors, parse, persist—and identify which steps may run concurrently and which must be serialized.
- **Staff:** describe fairness and capacity across servers, failure isolation, correlation IDs/run IDs, idempotent reruns, and how you would evolve orchestration as campaign volume grows. Separate evidenced behavior from proposed reliability mechanisms.

Evidence boundary. The project confirms parallel execution and structured persistence, but it does not document `asyncio`, a queue product, a transactional outbox, cursor pagination, or specific database technology. Those are design options to discuss, not implementation claims.

Scenario 2: storage roles and service boundaries in BenchOps Copilot

Project scenario. DPDK BenchOps Copilot used FastAPI as its service layer, Postgres for structured authoritative data, S3/MinIO for artifacts, a vector database for semantic representations, and MCP tools for `RunQuery`, `LogFetch`, `RunDiff`, and `CommandBuilder`.

How the concepts apply. The storage split follows access patterns: structured runs and metadata need authoritative queryable records; logs and large artifacts fit object storage; embeddings need similarity search. The tool boundary acts like an adapter/facade over deterministic business capabilities so the orchestration layer does not issue arbitrary database, shell, or storage commands.

Decision and trade-offs. Separating truth storage from semantic indexing adds operational components and consistency/versioning work, but it avoids treating a vector index as the system of record. Narrow tools reduce flexibility compared with generic SQL or shell access, but make validation, authorization, audit, and testing tractable.

Senior/Staff interview framing.

- **Senior:** explain the request path and why each datum belongs in Postgres, object storage, or the vector database.
- **Staff:** connect the split to ownership, failure containment, security, lineage, and evolution. State the trigger for adding a new store or service rather than presenting multiple technologies as intrinsically better.

9. Interview questions

1. Why return `202 Accepted` for an agent run or ingestion job?
2. How does a transactional outbox close the database/queue dual-write gap?
3. Why must consumers remain idempotent?
4. When do cursor and offset pagination differ?
5. Why choose relational storage for run state?
6. When is Redis appropriate, and what must not depend on it alone?
7. What happens when blocking code runs in an async endpoint?
8. When do async, threads, processes, and queues fit?
9. Why can a sliding-window nested loop still be linear?
10. When should BFS be preferred to DFS?
11. How does a visited set prevent graph failures?
12. Why does one-dimensional 0/1 knapsack iterate backward?
13. Why can a database rollback not undo an external side effect?
14. How does topological sorting reveal a dependency cycle?

10. Exit checklist

- [ ] Apply the architecture patterns to AI provider and retrieval boundaries.
- [ ] Design an idempotent async API with cursor pagination and stable errors.
- [ ] Choose the correct persistence system from access patterns.
- [ ] Explain event loop, task, timeout, semaphore, thread, process, and queue.
- [ ] Explain outbox, at-least-once delivery, and state transitions.
- [ ] Recognize all major DSA patterns and state complexity.
- [ ] Connect graphs/DAGs to pipelines and agent workflows.
- [ ] Distinguish transaction rollback, compensation, and reconciliation.

Source notes

- System Design Interview Coaching
- Async and Concurrency
- DSA Patterns
- DSA Core II
- Trees, Graphs, and DP
- GenAI Design Patterns

- Databases for AI Systems
- Deploying ML Models API
- Capstone Revision Day 1
- DPDK Automation for Network Packet Processing
- DPDK BenchOps Copilot